CentraleSupélec

Formalizing Hardware Security Mechanisms

Pierre Wilke

SUSHI, CentraleSupélec Rennes

Nov. 13th, 2025

CentraleSupélec

Overview

The security of systems eventually relies on the security of the hardware.
Goal : implement and prove hardware security mechanisms

- shadow stack, privilege levels, ...

Approach :

- we use Kôika, a Hardware Description Language embedded in Rocq
- we extend a RISC-V processor with security mechanisms
- we prove properties of these mechanisms

CentraleSupélec

Outline

1. Kôika
2. Implementation of hardware security mechanisms
3. Proving properties on Kôika models

CentraleSupélec

Outline

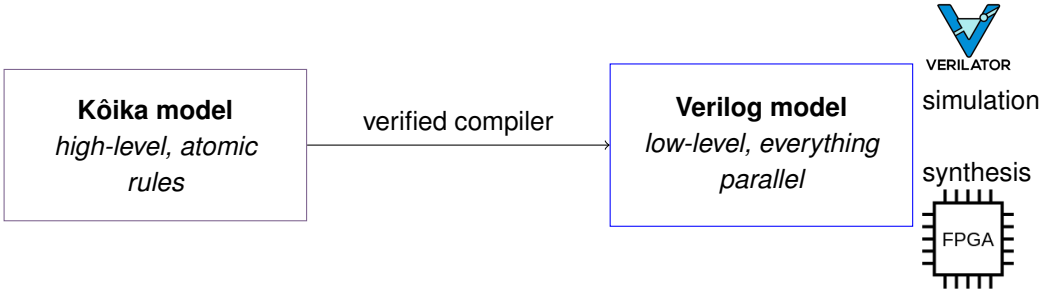
1. Kôika
2. Implementation of hardware security mechanisms
3. Proving properties on Kôika models

Hardware Design Workflow



Kôika (PLDI'20, *The Essence of BlueSpec*)

A Hardware Description Language embedded in Rocq.



Rules describe how the registers are updated **at each cycle**.

Conflicts occur e.g. when the same register is updated by two different rules.
In case of a conflict, the later rule is cancelled entirely.

Kôika

- Kôika programs are sets of rules, together with a scheduler
- Rules update **registers**
- Conflicts appear when :
  - reading a register *r* after a write on that register has occurred
  - writing on register *r* after a read or write on that register has occurred
- All rules execute during each cycle, however :
  - each rule may or may not contribute to the next state of registers, depending on whether conflicts appear

- Quite hard to predict whether a conflict will happen, hence whether a rule will succeed or fail...

CentraleSupélec

Outline

1 Kôika

2 Implementation of hardware security mechanisms

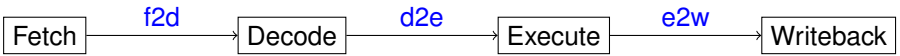
3 Proving properties on Kôika models

CentraleSupélec

A RISC-V processor in Kôika

Kôika developers provide an example model of a RISC-V processor

- 4-stage processor (Fetch, Decode, Execute, Writeback)
- RV32I
- unprivileged specification, no interrupts
- under 1000 lines of Kôika code
- runs on an actual FPGA board (TinyFPGA, LambdaConcept ECPIX-5)



CentraleSupélec

Implementation of hardware security mechanisms

We extended the RISC-V processor with :

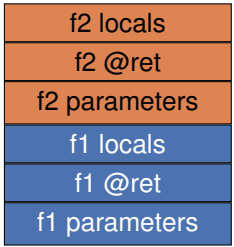
- traps (interrupts + exceptions)
- privilege levels (Machine and User)
- a **hardware shadow stack** + landing pads (Zicfilp)
- PMP (Physical Memory Protection, Smepmp)

Shadow stack

- Protection against buffer overflows that overwrite the return address
- Enforces (part of) control-flow integrity (only backward edges)
  - i.e., when we execute a `ret` instruction, we always jump back to (just after) our call site

Principle :

- when a `call` instruction is encountered, push `next(pc)` on the shadow stack
- when a `ret` instruction is encountered, pop `addr_ss` from the shadow stack and pop `addr` from the normal stack
  - If `addr_ss == addr`, continue
  - Else, we detect a violation





Shadow stack

Implementation :

- new memory region for our shadow stack
- instrument the `Execute` stage to push onto and pop from the shadow stack when needed
- when a violation is detected, we **raise a hardware exception**

What we want to prove

- Return to a **modified return address** $\Rightarrow$ exception
  - A bit more precisely :
    If the instruction about to be executed in the pipeline is a `ret` [1],
    and the address stored at the top of the shadow stack is different from the address to which we are about to jump,
    then an appropriate exception is raised.
- **Underflow or overflow** of the shadow stack $\Rightarrow$ exception
- Otherwise, behaviour **preserved**

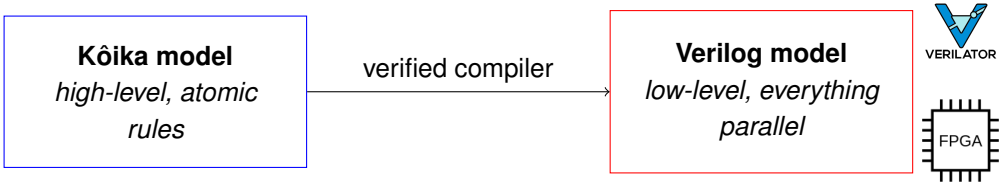
1. In RISC-V, `ret` is actually `jr ra`, i.e. jump to address contained in register `ra`.

CentraleSupélec

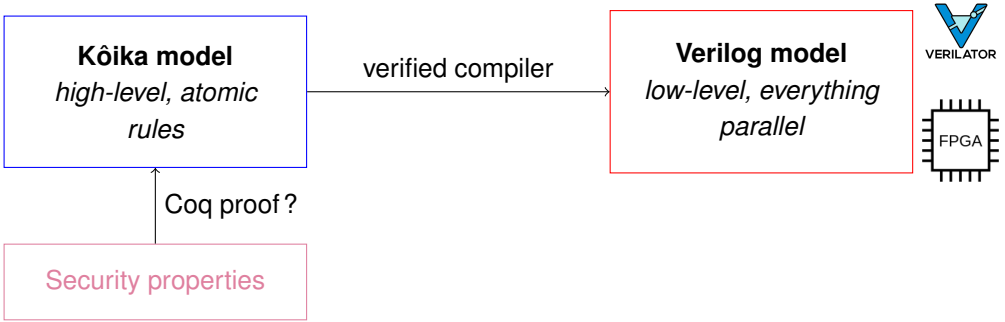
Outline

1. Kôika
2. Implementation of hardware security mechanisms
3. Proving properties on Kôika models

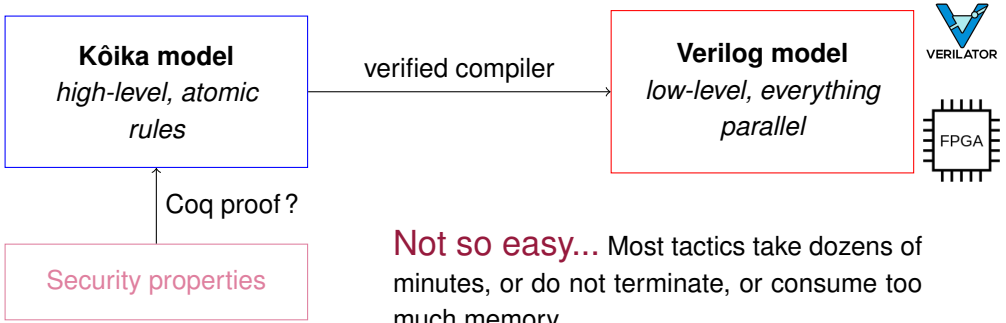
Proving properties on Kôika models



Proving properties on Kôika models



Proving properties on Kôika models



Not so easy... Most tactics take dozens of minutes, or do not terminate, or consume too much memory.

Proofs on Kôika models using an Intermediate Representation

Our solution : (CSF'23) we compile high-level Kôika models into **intermediate representations for reasoning** (IRR), more amenable to proofs.



$$e ::= v \mid cst \mid \texttt{reg}(r) \mid \triangleright e \mid e_1 \bowtie e_2 \mid \textbf{if } e_1 \textbf{ then } e_2 \textbf{ else } e_3$$
$$irr ::= \{ vars : V \to e \ ; \ final_values : \texttt{Reg} \to V \}$$

The **IRR** is a map $\texttt{Reg} \to \texttt{Expr}$, which gives the value of each register at the end of a cycle, depending on the values of registers at the beginning of the cycle.
In particular, the **conflict detection** logic is embedded into these expressions.

CentraleSupélec

Intermediate Representation for Reasoning (IRR)

Computing an IRR is quick ; but produces a large number of quite deep expressions.

We developed a range of **program transformations** akin to compiler optimizations on IRRs :

- constant folding ($3 + 4 \rightsquigarrow 7$)
- replace sub-expression e with e' (with a manual proof obligation that $e \equiv e'$)
- exploit partial information about register values (e.g. bits `6:0` of register `inst` are `0001101`) (with a manual proof obligation)

It's up to the (human) prover to apply each program transformation manually and prove the obligations.

CentraleSupélec

Correctness proofs of the shadow stack

Proof	No. lines of proof	`Time Qed.`
Underflow $\Rightarrow$ exn	150	1m10s
Overflow $\Rightarrow$ exn	270	2m30s
Wrong address $\Rightarrow$ exn	900	3m10s

Lots of lines of boring proofs, very fragile (numbering of variables).

Our proofs are mainly case studies about bitvectors. We can discharge these proofs to SMT solvers !

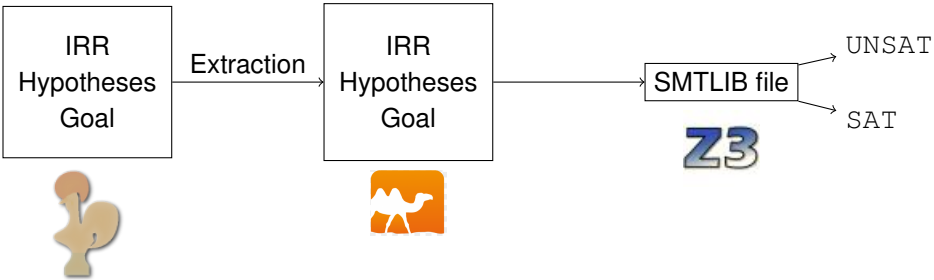
CentraleSupélec

Example SMT encoding

```
; all variables in IRR
(assert (= v_1 (encode_expr e_1)))
...
(assert (= v_n (encode_expr e_n)))

; hypotheses
; sstack_full
(assert (= reg_shadow_stack_size capacity))
; the instruction about to be executed is a call
(assert ...)
; assert negation of goal
(assert (not (has_exn)))
(check-sat) ; expect unsat
(get-model)
```

Proofs workflow



Proof	No. lines of proof	`Time Qed.`	Time `z3` (s)
Underflow ⇒ exn	150	1m10s	1.20
Overflow ⇒ exn	270	2m30s	1.20
Wrong address ⇒ exn	900	3m10s	1.22
No problem			26

Better integration, with SMTCoq ?

OK but we're leaving the Rocq world... Can we keep all the formal guarantees of Rocq **and** the automation provided by SMT solvers ?

SMTCoq [2] sounds like a possible solution

- transforms current goal into a SMT formula
- if `unsat` : solver generates an unsat core, translated back into a Rocq proof
- if `sat` : solver generates an (counter-)example model, given back to user

To be investigated !

2. `https://github.com/smtcoq/smtcoq`

Conclusion

Done :

- implementation of hardware security mechanisms
  - traps, privilege levels, shadow stack, landing pads, PMP
- SMT-backed proofs of these mechanisms

Just started, 2 PhDs :

- exploration of countermeasures against fault injections (with M. Jan and D. Couroussé, CEA)
- more security mechanisms (enclaves, MMU) and interaction with software (compiler and OS)

Next :

- integration with SMTCoq
- functional correctness wrt. an ISA specification (Sail)
- investigate mechanisms at the HW/SW interface (e.g. memory protection, enclaves)

pierre.wilke@centralesupelec.fr